

Lab 2 TCP Attacks and Detection

Network Security (50.020) | Timothy Seah | Student ID 1007654 | Experiment: 1 October 2026

node1 client 10.11.2.5, node2 server 10.11.2.6, and node3 attacker 10.11.2.7.

1. Network and basic operations

Figure 1 records the IP addresses of all three nodes. Node2 reached node1 and node3 without packet loss, and Wireshark captured network traffic. Figure 2 shows the three containers running.



Figure 1. IP addresses checked on node1, node2, and node3.



2. TCP SYN flooding

Parameters and method

Parameter	Node2 value shown	Purpose
tcp_max_syn_backlog	128	Limits pending connection requests before the handshake completes
tcp_syncookies	0 initially	When enabled, SYN cookies help the server respond under SYN-queue pressure without retaining pending state for every request
tcp_synack_retries	5	Controls how many SYN-ACK retries are sent while awaiting the final ACK

At baseline, node2 showed tcp_max_syn_backlog=128, tcp_syncookies=0, and tcp_synack_retries=5. The Kali host readings were 256, 1, and 5 respectively. The host allowed a larger pending queue and had SYN cookies enabled, while both systems used five SYN-ACK retries.

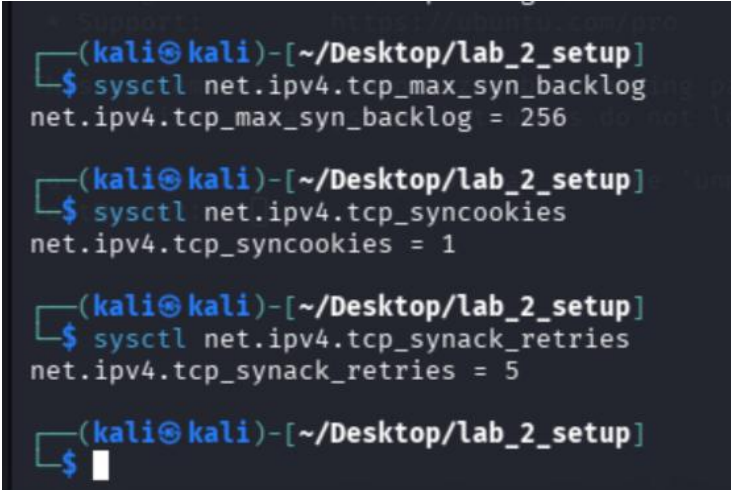


Figure 3. Kali-host TCP values used for comparison with node2.

A normal Telnet connection completed the SYN, SYN-ACK, ACK handshake. After closing it, I ran `ip tcp_metrics flush` to clear stored information about previous connections to the peer. This made the next run easier to compare but did not clear the SYN backlog.

In the first flood run, node3 sent SYN packets to Telnet port 23 from changing spoofed addresses without completing the handshakes. Node2 recorded 97 connections in SYN-RECV. Node1 retransmitted its SYN four times before receiving a SYN-ACK, then completed login. The flood delayed this connection but did not cause a sustained outage.

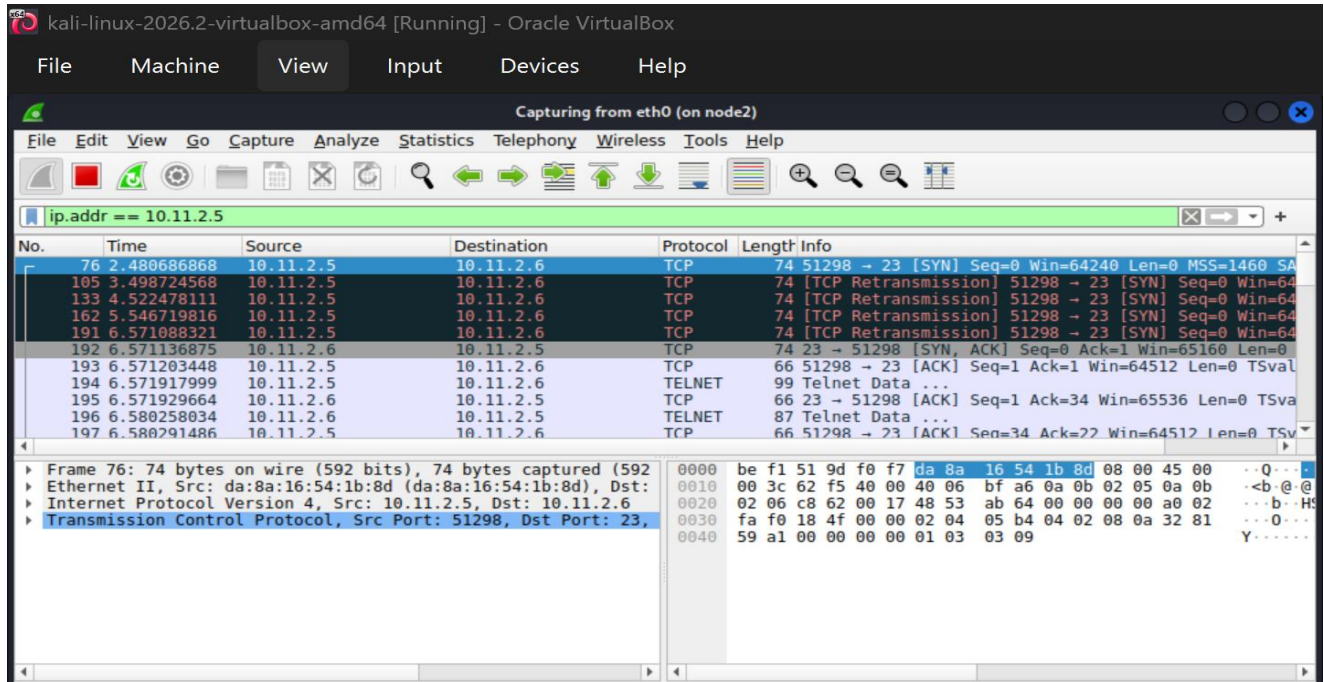


Figure 4. Node1 retransmitted its Telnet SYN four times during the flood.

A later capture from all three nodes shows the traffic from each position. Node3 recorded SYN packets with changing spoofed source addresses sent to node2:23. Node2 captured those SYNs alongside node1's Telnet traffic. Node1 captured its own Telnet handshake, which completed in that run. The earlier run above shows the connection delay.

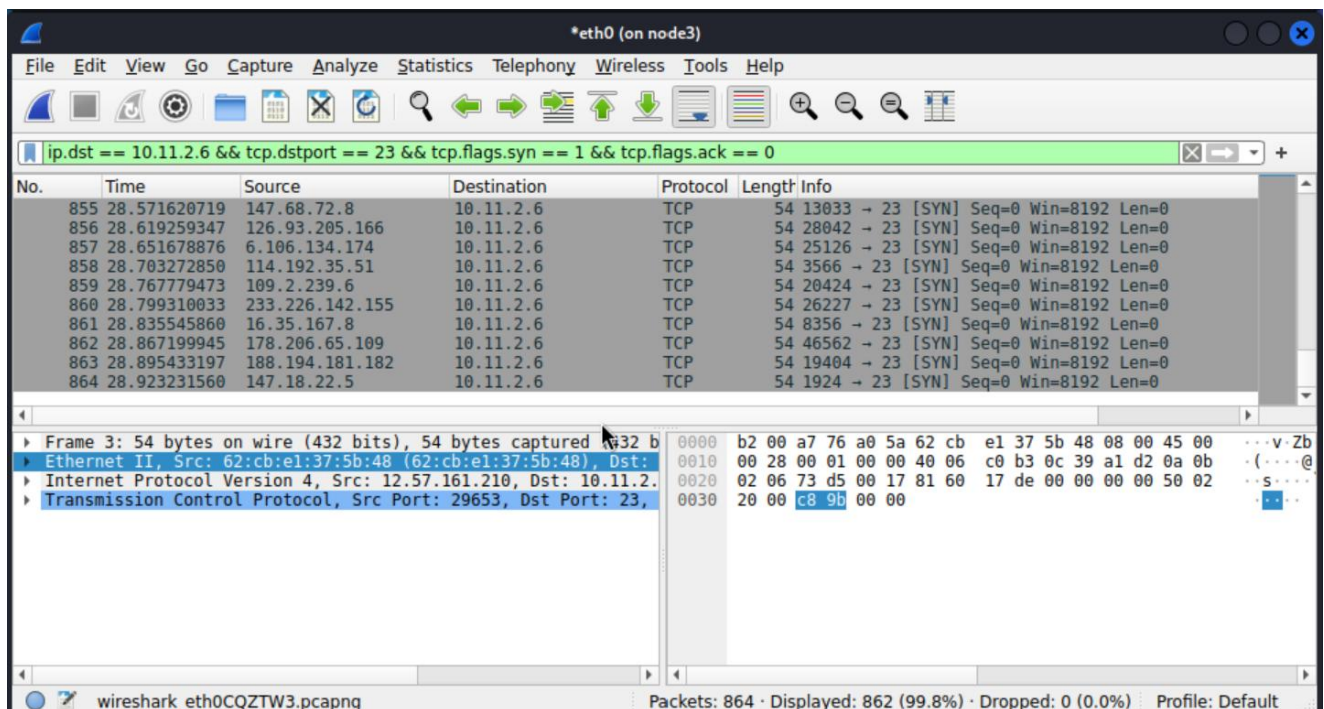


Figure 5. Node3 sends SYN packets to node2:23 with changing spoofed source IPs.

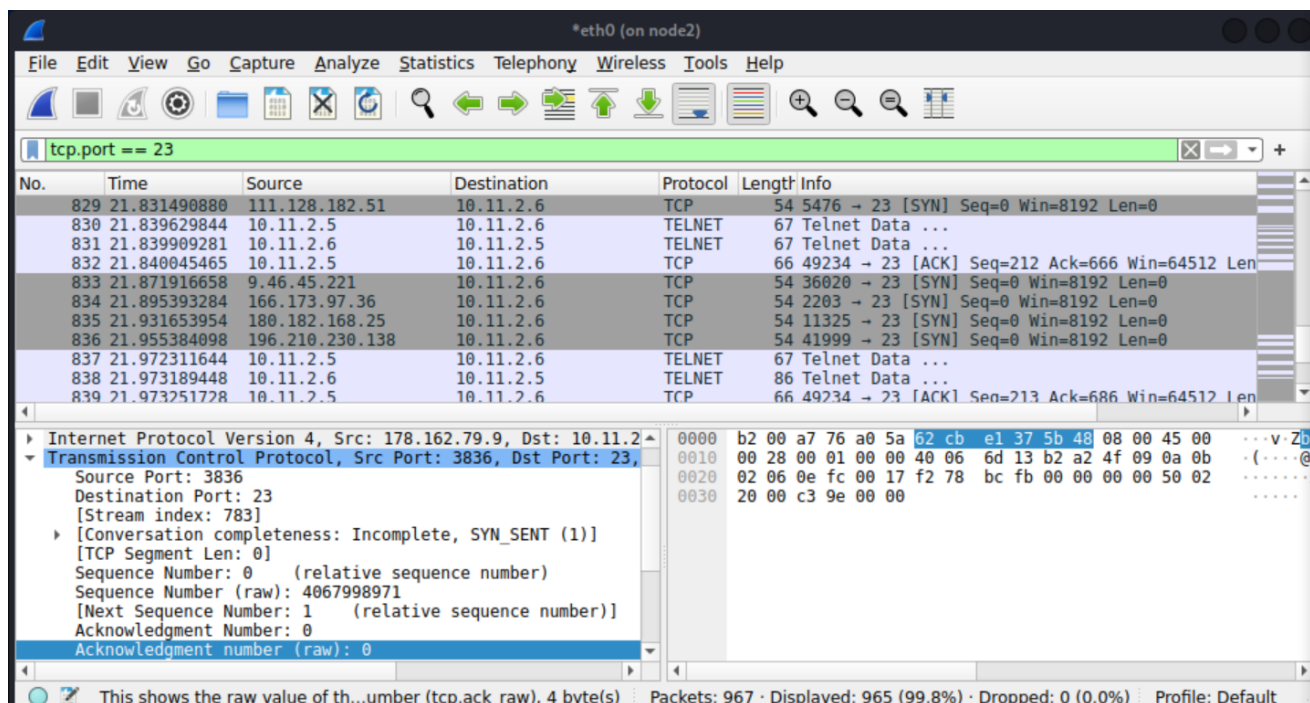


Figure 6. Node2 captures spoofed SYNs and node1's Telnet traffic.

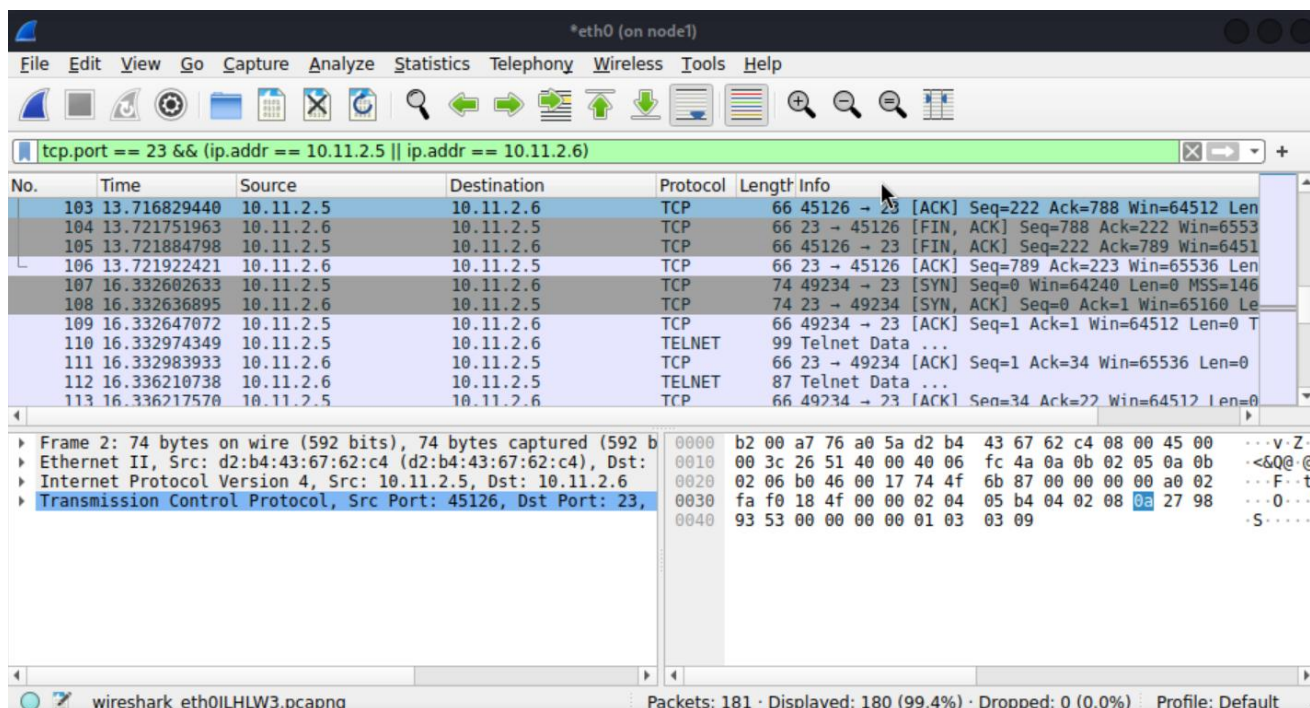


Figure 7. Node1 captures its own Telnet handshake in the later run.

Mitigation

On node2, `net.ipv4.tcp_syncookies` was changed from 0 to 1 and read back as 1 (Figure 8). During a renewed flood, the half-open count shown reached 127. Node1 completed a new Telnet handshake and login (Figure 9), showing that the service remained available in this run.

```

root@node2: /

Session Actions Edit View Help

net.ipv4.tcp_synack_retries = 5
root@node2:~# wireshark
** (wireshark:377) 06:45:20.547767 [GUI WARNING] -- QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/ru
ntime-kali'
** (wireshark:377) 06:45:23.478827 [Capture MESSAGE] -- Capture Start ...
** (wireshark:377) 06:45:23.526992 [Capture MESSAGE] -- Capture started
** (wireshark:377) 06:45:23.527097 [Capture MESSAGE] -- File: "/tmp/wireshark_eth09C6BW3.pcapng"
** (wireshark:377) 06:45:50.270480 [Capture MESSAGE] -- Capture Stop ...
** (wireshark:377) 06:45:50.323035 [Capture MESSAGE] -- Capture stopped.
** (wireshark:377) 06:45:50.323128 [Capture WARNING] ./ui/capture.c:722 -- capture_input_closed():
** (wireshark:377) 06:45:50.445043 [Capture MESSAGE] -- Capture Start ...
** (wireshark:377) 06:45:50.560958 [Capture MESSAGE] -- Capture started
** (wireshark:377) 06:45:50.561016 [Capture MESSAGE] -- File: "/tmp/wireshark_eth097DBW3.pcapng"
^CInterrupt
root@node2:~# ip tcp_metrics flush
root@node2:~# ip tcp_metrics flush
root@node2:~# watch -n 0.2 'ss -Hnt state syn-recv | wc -l'
root@node2:~# ip tcp_metrics flush
root@node2:~# ip tcp_metrics flush
root@node2:~# watch -n 0.2 'ss -Hnt state syn-recv | wc -l'
root@node2:~# ip tcp_metrics flush
root@node2:~# watch -n 0.2 'ss -Hnt state syn-recv | wc -l'
root@node2:~# ip tcp_metrics flush
root@node2:~# sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1
root@node2:~# sysctl net.ipv4.tcp_syncookies
net.ipv4.tcp_syncookies = 1
root@node2:~#

```

Figure 8. SYN cookies enabled and verified on node2.

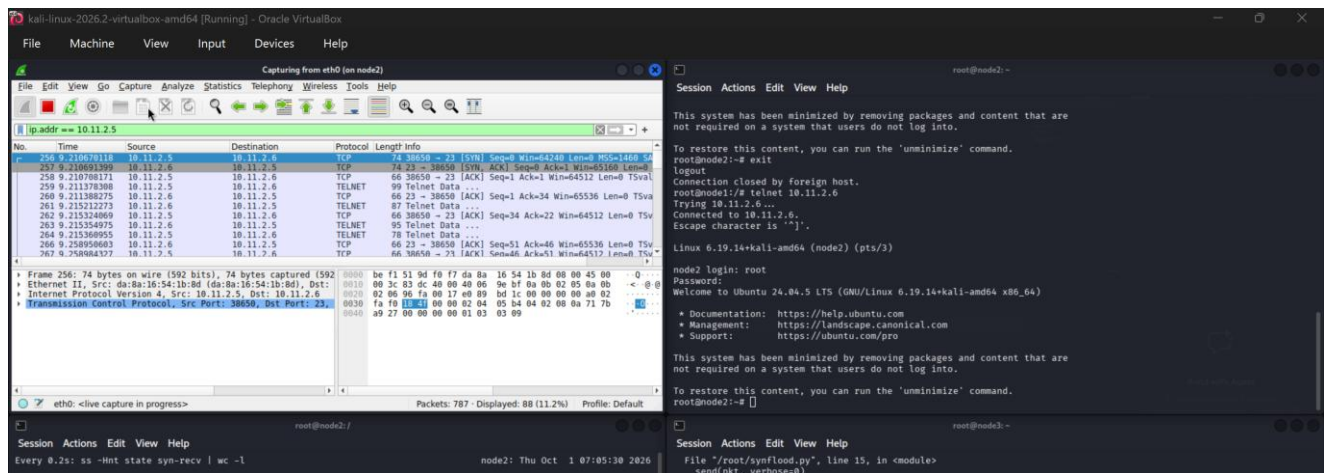


Figure 9. Node1 completes Telnet login while the renewed flood runs.

3. TCP reset attack

For the active SSH flow, node3 forged a packet claiming source 10.11.2.5:51458 and destination 10.11.2.6:22, with RST set and raw TCP sequence 2264913361 (relative sequence 4128). Wireshark captured a forged RST matching the SSH session's addresses and ports, with a sequence number the receiver would accept. The packet had no ACK flag or payload, and node1's SSH client reported "Broken pipe".

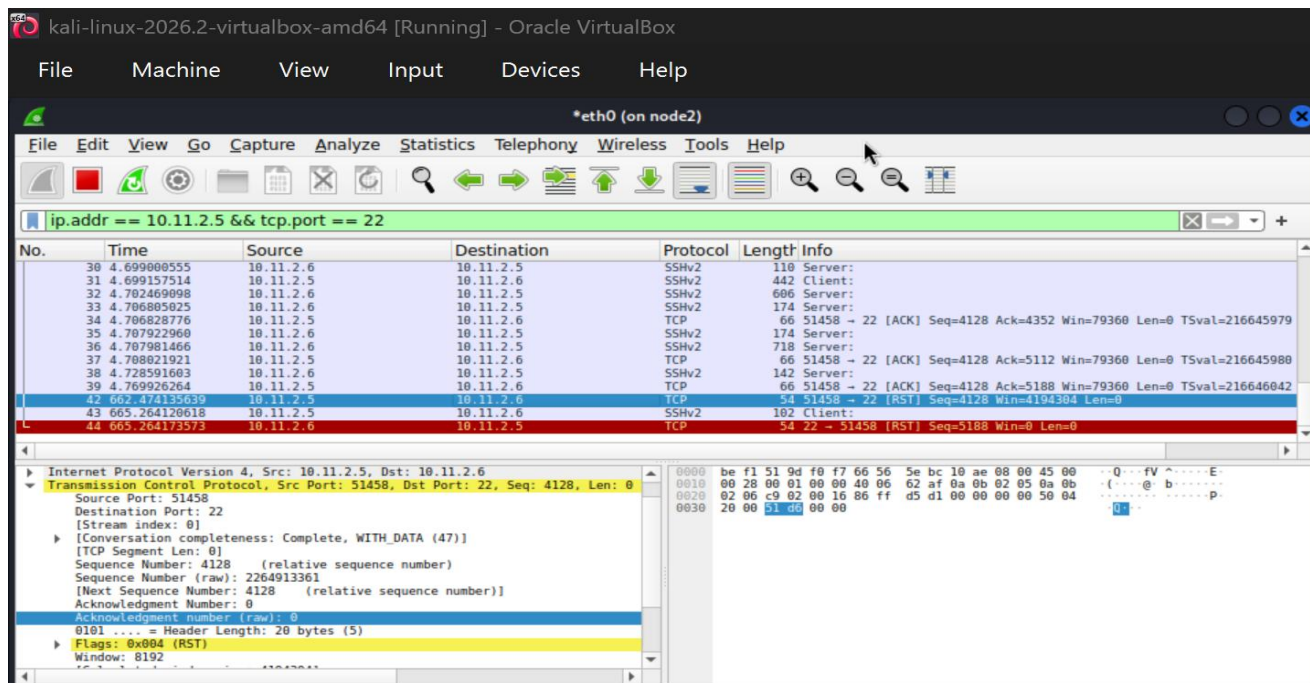


Figure 10. Forged RST in the SSH flow and the client disconnected.

An attacker restricted to node3 could still spoof an IP packet, but an external attacker may not know the live client port and next sequence number. They would need visibility into the session or another way to infer those values. Simply knowing the two IP addresses is insufficient.

4. TCP session hijacking

While node1's root Telnet session was open, node3 sent a forged PSH, ACK packet to node2 on the same connection (10.11.2.5:42420 → 10.11.2.6:23). It used the observed raw sequence number 636491331 and acknowledgment number 17698764, with touch /root/log.txt\r\n as its 21-byte payload. Node2's relative acknowledgment moved from 208 to 229, and /root/log.txt appeared. The server's later retransmissions suggest the two endpoints had fallen out of sync after the injection.

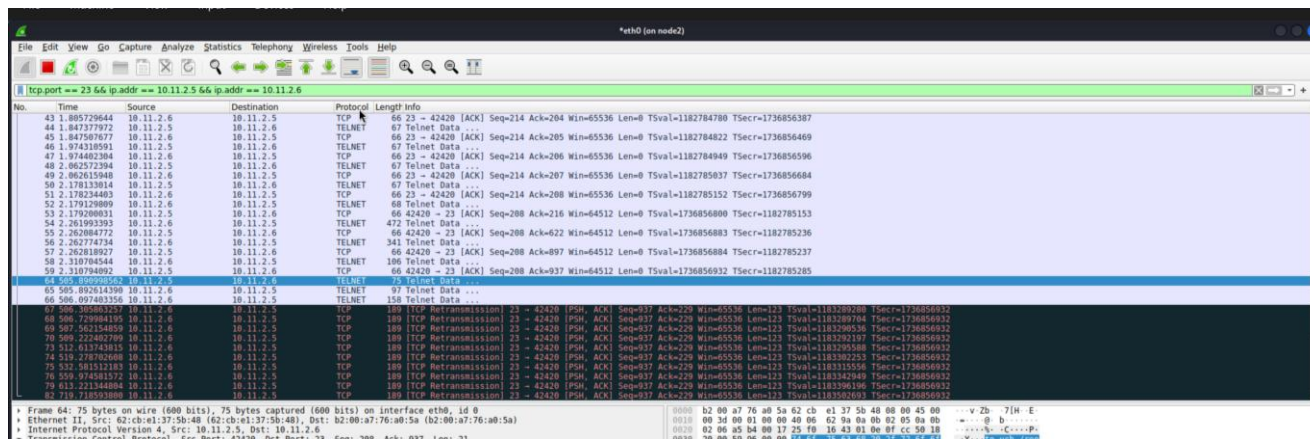


Figure 11. Injected Telnet packet with a 21-byte command payload.

```
root@node2:/# ls -l /root/log.txt
ls: cannot access '/root/log.txt': No such file or directory
root@node2:/# exit
exit

(kali@kali)-[~]
$ docksh node2
root@node2:/# ls -l /root/log.txt
ls: cannot access '/root/log.txt': No such file or directory
root@node2:/# ls
bin          boot  etc  lib          lib64  mnt  proc  run  sbin.usr-is-merged  srv  tmp  var
bin.usr-is-merged  dev  home  lib.usr-is-merged  media  opt  root  sbin  shared  sys  usr
root@node2:/# cd root/
root@node2:~# ls
root@node2:~# ls -l /root/log.txt
-rw-r--r-- 1 root root 0 Oct 1 08:13 /root/log.txt
root@node2:~# ls
log.txt
root@node2:~#
```

Figure 12. Node2 lists /root/log.txt after the injection.

For the controlled reverse-shell test, node3 listened on TCP port 4444 while node1's Telnet session remained open. The forged Telnet command made node2 connect back to node3. The listener showed a root@node2 prompt, hostname returned node2, and id reported uid=0(root).

```
Listening on 0.0.0.0 4444
Connection received on 10.11.2.6 54190
root@node2:~# hostname
hostname
bash: hostname: command not found
root@node2:~# hostname
hostname
node2
root@node2:~# id
id
uid=0(root) gid=0(root) groups=0(root)
root@node2:~#
```

Figure 13. Node3's listener receives the node2 shell and confirms its identity.

5. AI-assisted real-time attack detection

Key prompts. I gave the AI tool the Task 5 requirements and asked, “Now create a script for Task 5,” followed by, “Ok walk me through from start to end for Task 5.” I checked the proposed detection logic against the three earlier attacks, then ran tcp_attack_detector.py on node2’s eth0 interface.

Attack	Detection approach	Observed result
SYN flood	Count SYNs and unfinished handshakes in five-second windows. Alert at 20.	Alerts at 20 and 170 SYNs from the same number of source IPs. Pending connections reached 455.
SSH RST	Check the Ethernet source and TCP timestamp against the SSH connection.	SUSPICIOUS SSH RST: Flagged a RST from port 58738 with a different Ethernet source and no timestamp.
Telnet injection	Check Telnet data for those differences and the file creation command.	POSSIBLE TELNET INJECTION: Flagged a 21-byte packet on port 41914 with both differences and the command marker.

```

root@node2:~# cp /shared/tcp_attack_detector.py /root/
root@node2:~# ls
log.txt  tcp_attack_detector.py
root@node2:~# python3 tcp_attack_detector.py
Watching eth0 on node2 (10.11.2.6); start SSH/Telnet sessions now. Ctrl+C stops.
Learned 10.11.2.5 Ethernet address d2:b4:43:67:62:c4 from SYN
[2026-10-01 08:43:36+0000] SYN FLOOD: 20 SYNs in 5s, 20 source IPs, 20 incomplete handshakes; latest destination port 23
[2026-10-01 08:43:41+0000] SYN FLOOD: 170 SYNs in 5s, 170 source IPs, 173 incomplete handshakes; latest destination port 23
[2026-10-01 08:43:46+0000] SYN FLOOD: 167 SYNs in 5s, 167 source IPs, 325 incomplete handshakes; latest destination port 23
[2026-10-01 08:43:51+0000] SYN FLOOD: 142 SYNs in 5s, 142 source IPs, 455 incomplete handshakes; latest destination port 23
[2026-10-01 08:44:14+0000] SYN FLOOD: 1 SYNs in 5s, 1 source IPs, 248 incomplete handshakes; latest destination port 22
[2026-10-01 08:46:43+0000] SUSPICIOUS SSH RST: 10.11.2.5:58738 → 10.11.2.6:22; Ethernet source 62:cb:e1:37:5b:48 differs from expected d2:b4:43:67:62:c4; TCP timestamp missing from timestamp-enabled flow
[2026-10-01 08:49:15+0000] POSSIBLE TELNET INJECTION: 10.11.2.5:41914 → 10.11.2.6:23, 21 payload bytes; Ethernet source 62:cb:e1:37:5b:48 differs from expected d2:b4:43:67:62:c4; TCP timestamp missing from timestamp-enabled flow; matched lab file-creation command

```

Figure 14. Node2 detector output contains all three alert classes.

Assessment. The detector raised real-time alerts for all three attacks tested in the lab. Its pending-flow count is an estimate and may differ from the kernel's SYN-RECV count. The SSH and Telnet alerts also need context because Ethernet addresses can change, legitimate RSTs occur, and an authorized user could type the same Telnet command. The command check only works when the traffic is visible as plaintext.